synchronization mechanism that periodically serialize our in-memory data structure into a persistent database.

5.3 Describing the Data Structure API: Disjoint Set

In our design, our data structure needs to offer only a few, though crucial, operations.

<table>
<tr><th colspan="2">Abstract Data Structure: <code>DisjointSet</code></th></tr>
<tr><td>API</td><td><pre>class DisjointSet {
        init(U);
        findPartition(x);
        merge(x, y);
        areDisjoint(x,y);
}</pre></td></tr>
<tr><td>Contract with client</td><td>A disjoint set keeps track of the mutual relations between elements in a universe U.
The relation is not explicitly defined by the data structure, it is left to the client to define it.
However, it is assumed that such a relation ® has the reflexive, symmetric and transitive properties: this means that, given x, y, z elements of U
· x®x
· if x®y, then y®x
· if x®y and y®z then x®z
The guarantees provided by the class are:
1) It’s possible to add a relation between any two elements.
2) If two elements at any point are merged (i.e. a relation between</td></tr>
</table>